

LEARNER NOTES

ML Module 1 — Traditional Machine Learning

From Machine Learning Fundamentals to Regularized Regression

For Data Science / ML consultants · Examples set in London · Python + scikit-learn

1. What is Machine Learning?
2. How Does ML Work?
3. Types of Machine Learning
4. Where Does Regression Fit?
5. Basic ML Workflow
6. Correlation Analysis
7. Correlation Matrix
8. From Correlation to Regression
9. Simple Linear Regression
10. Python: Simple Linear Regression
11. Multiple Linear Regression
12. Model Evaluation
13. Actual vs Predicted
14. Generalization
15. Underfitting & Overfitting
16. Bias and Variance
17. Why Regularization?
18. Ridge Regression
19. Lasso Regression
20. Ridge vs Lasso
21. Hands-on Lab
22. Model Comparison
23. ML Module 1 Recap

START HERE

01

What is Machine Learning?

Let's start with a question every client eventually asks: "*Can we predict this?*" Machine Learning is how we answer it — not by writing rules, but by learning from examples.

A simple example: London flat prices

HOUSE SIZE	PRICE
500 sq ft	£300k
750 sq ft	£450k
1,000 sq ft	£600k
875 sq ft	?

🧠 What does your brain do?

You look at the examples, spot a pattern (*every extra 250 sq ft adds about £150k — roughly £600 per sq ft*), and guess the missing price — roughly **£525k**.

That is exactly what Machine Learning does — just with thousands of rows and many more columns.



CORE IDEA

Machine Learning learns patterns from existing data and uses those patterns to make predictions or decisions on new data.

Traditional Programming vs Machine Learning

Flip between the two approaches

Traditional Programming



A developer **writes the rule** by hand:

```

if area > 1200: price = "high"
price = area × £600      # rule someone had to figure out
  
```

Works when rules are known and stable. Breaks when the pattern is complex (a London price depends on area, borough, Tube access, age, schools, demand...) or keeps changing.

Machine Learning



We give the algorithm **past examples with known answers**. It **figures out the rule itself** and stores it as a *model*.

Works when the pattern is too complex to hand-code, or when we have lots of historical data. The "rule" is learned, not written.

	TRADITIONAL PROGRAMMING	MACHINE LEARNING
Who creates the logic?	The developer	The algorithm, from data
Input	Rules + data	Data + known answers
Output	Answers	A model (the learned rules)
Best for	Tax calculation, invoicing	Price prediction, fraud detection



LEADING TO THE NEXT CONCEPT

"Now that we understand what ML is, the next question is: how does a machine actually learn from data?"

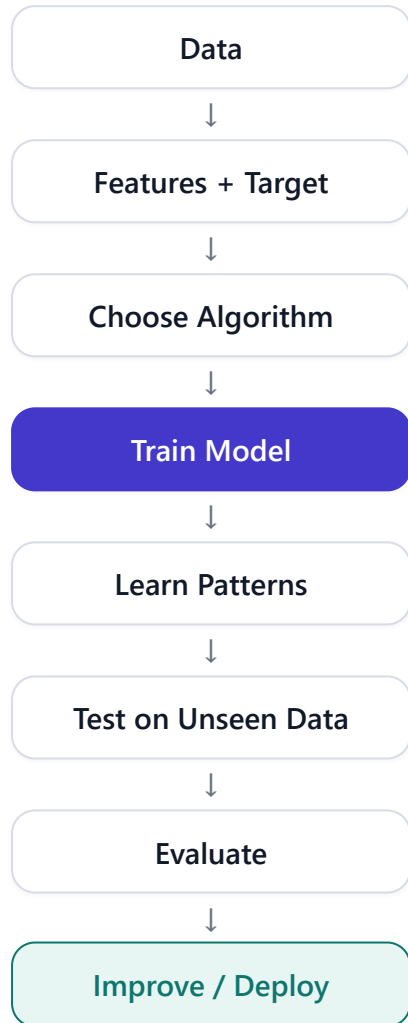
THE LEARNING CYCLE

02

How Does Machine Learning Work?

**WHY ARE WE LEARNING THIS NOW?**

We know *what* ML is. Before touching any algorithm, we need the big picture of *how* learning happens — every model today follows this cycle.



Click any step to see what happens there.

Start with **Data** — historical records where we already know the answer.

Two words you'll hear throughout this module

Features (X) — the inputs we use to predict.
e.g. experience

Target (y) — the value we want to predict. *e.g. salary*

Example: predicting a London analyst's salary from experience

Try it: change experience, watch the prediction

INTUITION DEMO — NOT A REAL TRAINED MODEL

EXPERIENCE	SALARY
1 year	£30k
3 years	£42k
5 years	£55k
8 years	£75k

6 yrs

→

Model

→

£62k



The line is the "pattern" learned from the 4 examples. A new input lands on that line → that's the prediction. Notice how predictions far outside 1–8 years (e.g. 12 years) are less trustworthy — the model never saw such data.

 Sketch pad — explain “How Does ML Work?” by hand

saved notes

I/p feature $\rightarrow$ E	Salary $\rightarrow$ Target
5	20k
7	35k
9	50k
3	15k (5, 20)
15 = ?	

Exp, Sal $\rightarrow$ 



LEADING TO THE NEXT CONCEPT

“We’ve seen a model learn from labelled examples. But is that the only way machines learn? Let’s look at the different types of ML.”

THE ML FAMILY

03

Types of Machine Learning

**WHY ARE WE LEARNING THIS NOW?**

Not every problem has "known answers" in the data. The type of data we have decides the type of ML we use.

👉 Click a card to explore each type.

**Supervised**

Learn from data with known answers

**Unsupervised**

Find hidden structure — no answers given

**Reinforcement**

Learn by trial, reward and penalty

**Supervised Learning****THIS MODULE**

Input + Known Output →

Learn →

Predict Output

Like a student with a solved answer key: the data contains the **target column**, so the model can check itself while learning.

**Regression****This module's focus**

Predict a **number** (continuous value)

House price · Salary · Sales · Demand

**Classification**

Predict a **category** (label)

Spam / Not spam · Fraud / Not fraud ·
Disease / No disease

⚡ CHECK 1

Predicting house price is:

A. Classification

B. Regression ✓ correct

C. Clustering

D. Reinforcement Learning

Answer: B. Regression — House price is a continuous number (£525k, £650k...), and we have past prices to learn from → supervised regression.

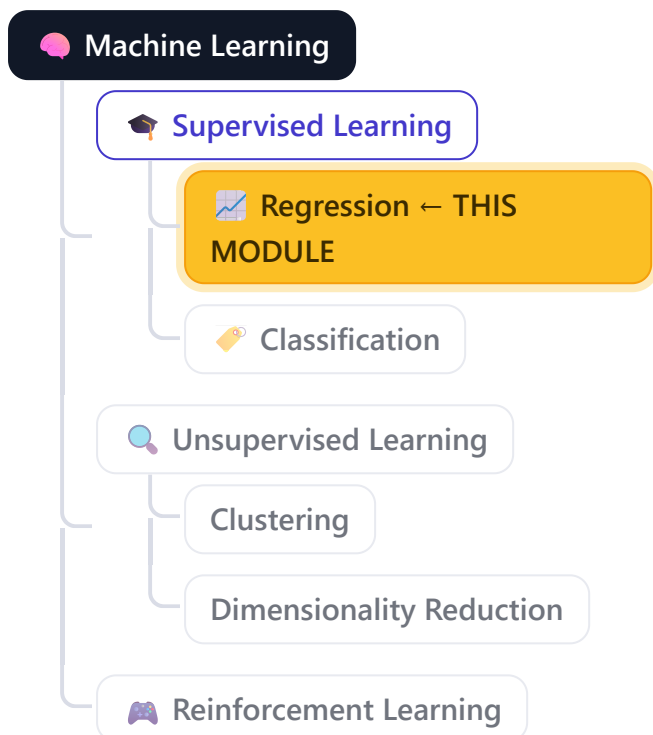
**LEADING TO THE NEXT CONCEPT**

"Now we know regression lives inside supervised learning. Let's zoom in and see exactly where it sits on the ML map."

THE MAP

04

Where Does Regression Fit?



DEFINITION

Regression is a supervised learning problem where the target is a continuous numerical value.

INPUT	REGRESSION OUTPUT
 Flat in Islington	£650k
 Data consultant	£75k salary
 High-street store (weekly)	10,500 sales
 Tube station (daily)	3,200 passengers



QUICK TEST

Ask: "Is the answer a number on a scale, or a label from a list?" Number → Regression. Label → Classification.



Sketch pad — explain “Where Does Regression Fit?” by hand

saved notes



LEADING TO THE NEXT CONCEPT

“Now we know where Regression fits. Before building anything, let's see the end-to-end workflow a real ML project follows.”

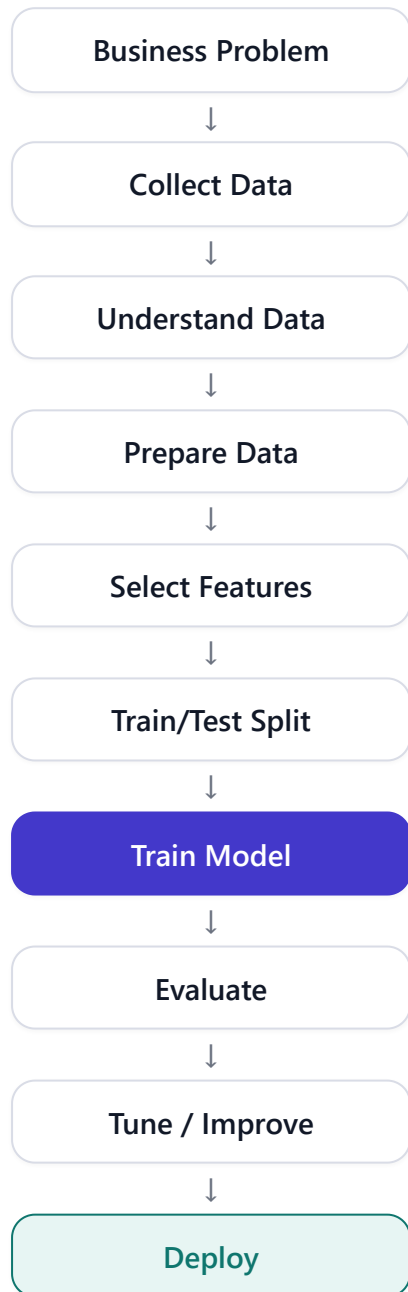
END-TO-END

05

Basic Machine Learning Workflow

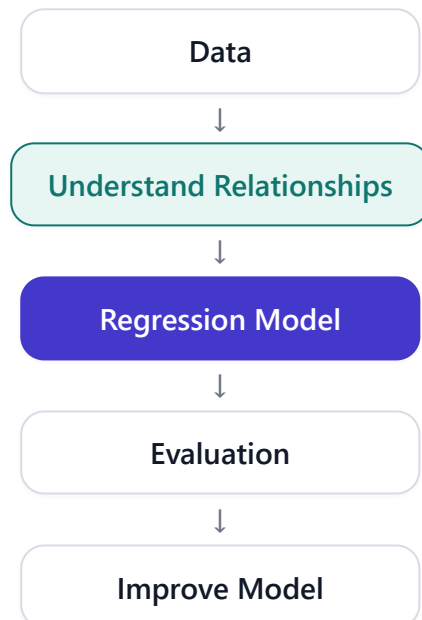
**WHY ARE WE LEARNING THIS NOW?**

A model is only one step. In consulting, most time goes into everything *around* the model. This is the route map for every project.

A real project

Click a step for a one-line explanation.

Every project starts with a **business problem**: "Can we estimate a fair price for a listing before it goes live?"

This module's zoomed-in workflow

**WHY UNDERSTAND DATA BEFORE MODELLING?**

A model can only learn what's in the data. If we don't know which columns relate to the target, which ones overlap, or which ones are junk, we'll build a model we can't explain or trust.

Garbage in → garbage out.

**LEADING TO THE NEXT CONCEPT**

"Before building a regression model, let's first understand the relationships between our variables."

RELATIONSHIPS

06

Correlation Analysis

**WHY ARE WE LEARNING THIS NOW?**

Before building a regression model, we first need to understand the relationships inside our data. Correlation is our first lens.

**DEFINITION**

Correlation measures the strength and direction of a linear relationship between two numerical variables.

**Positive**

Experience $\uparrow$ $\rightarrow$ Salary $\uparrow$
Both move together

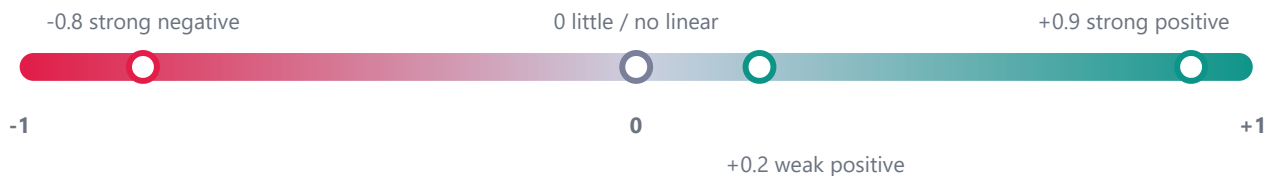
**Negative**

Distance from central London $\uparrow$ $\rightarrow$ Price $\downarrow$
One goes up, the other goes down

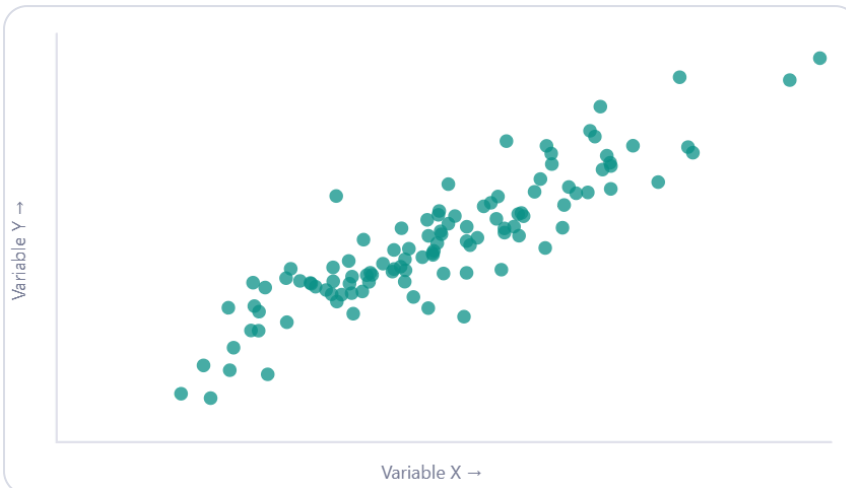
**None / weak**

Employee ID $\leftrightarrow$ Salary
No consistent pattern

The correlation coefficient (r)



Sign (+/-) = direction. Distance from 0 = strength. A rough guide: $|r| > 0.7$ strong, 0.3–0.7 moderate, < 0.3 weak.

Interactive: see correlation change the shape**SIMULATED DATA****CORRELATION R (COMPUTED)****+0.90**

strong positive linear relationship. As X rises, Y tends to rise.

⚡ CHECK 2**Which correlation represents the strongest positive relationship?**

A. -0.92

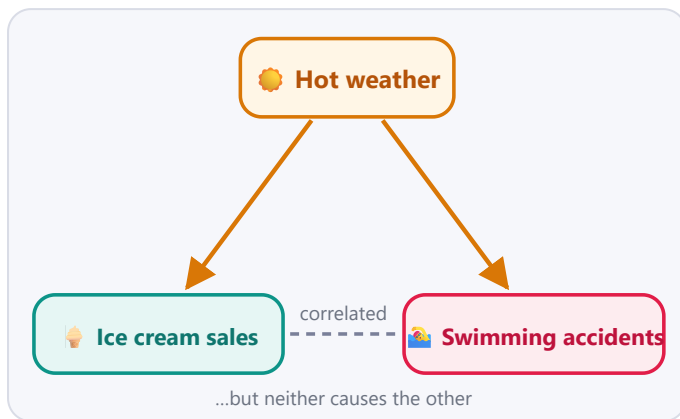
B. 0.12

C. 0.91 ✓ correct

D. 0.01

Answer: C. 0.91 — 0.91 is closest to +1, so it's the strongest positive relationship. -0.92 is strong too — but negative.

⚠️ Correlation does not imply causation



During a UK summer heatwave, ice cream sales and swimming accidents rise together in the data. Does ice cream cause drowning? Of course not.

A **third variable — hot weather** — drives both. More people buy ice cream *and* more people go swimming.



Correlation tells us two things move together. It does not tell us one causes the other. Always ask: "Is there a hidden third factor?"



LEADING TO THE NEXT CONCEPT

"Correlation compares two variables at a time. But real datasets have many columns. How do we see all the relationships at once?"

07

MANY VARIABLES AT ONCE

Correlation Matrix



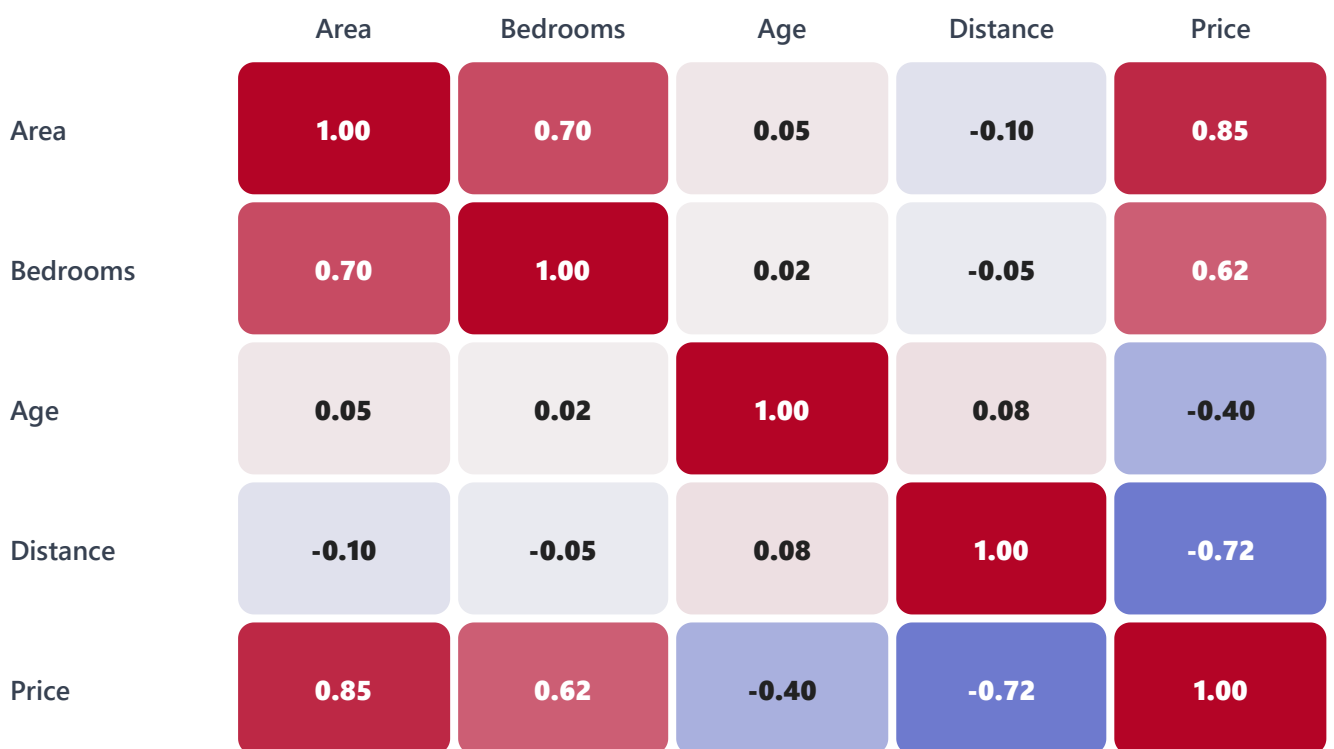
WHY ARE WE LEARNING THIS NOW?

With 5 columns there are 10 pairs to check; with 20 columns, 190. A **correlation matrix** shows every pair in one grid — and a heatmap makes it readable at a glance.

Example housing correlation heatmap

ILLUSTRATIVE VALUES

👉 Click any cell to read what it means.



-1 strong negative

0 none

+1 strong positive

Click a cell...

Strong +

Area ↔ Price (0.85)
— useful predictor

Strong -

Distance ↔ Price
(-0.72) — also
useful

Weak

Age ↔ Area (0.05)
— little linear link

Diagonal = 1

Every variable
correlates perfectly
with itself

Correlation table

PYTHON

```
df.corr(numeric_only=True)
```

Returns a table of r values for every pair of numeric columns. `numeric_only=True` skips text columns like `location`.

Correlation heatmap

PYTHON

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.show()
```

`annot=True` writes the number in each cell; `cmap="coolwarm"` makes negatives blue and positives red.

**USE IT WISELY**

Correlation can help us understand relationships and identify potentially useful or **redundant** features (e.g. Area ↔ Bedrooms = 0.70 — they carry overlapping information), but it should **not** be treated as the only feature-selection method. It only sees *linear, pairwise* relationships.

**LEADING TO THE NEXT CONCEPT**

"We have looked at relationships between variables. Now let's use those relationships to build a model that can make predictions."

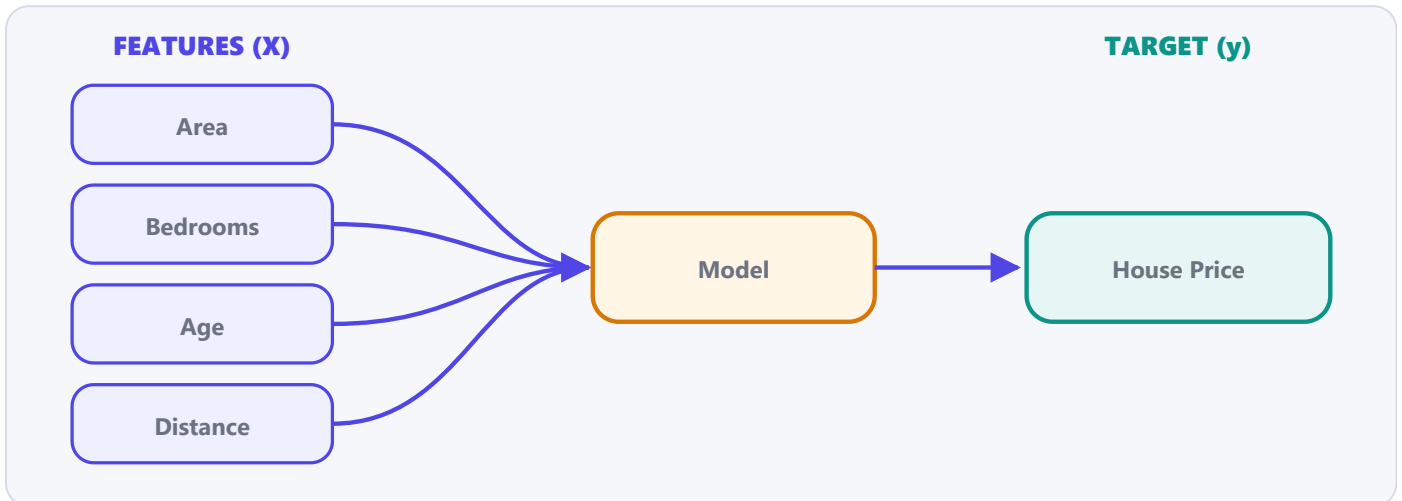
THE BRIDGE

08

From Correlation to Regression

**WHY ARE WE LEARNING THIS NOW?**

We have looked at relationships between variables. Now let's use those relationships to build a model that can make predictions. Correlation *describes*; regression *predicts*.

**X = Features**

The columns we **use** to make a prediction. Capital X because it's usually a table (many columns).

y = Target

The single column we **want to predict**. Lower-case y because it's one column.

Your turn: we want to predict the selling price. Click the target (y).

area

bedrooms

age

distance

price

Which column is the answer we want the model to produce?

**LEADING TO THE NEXT CONCEPT**

"Let's start with the simplest possible model: one feature, one target, one line."

ONE FEATURE, ONE LINE

09

Simple Linear Regression

**DEFINITION**

Simple Linear Regression uses one feature to predict one continuous target.

Area

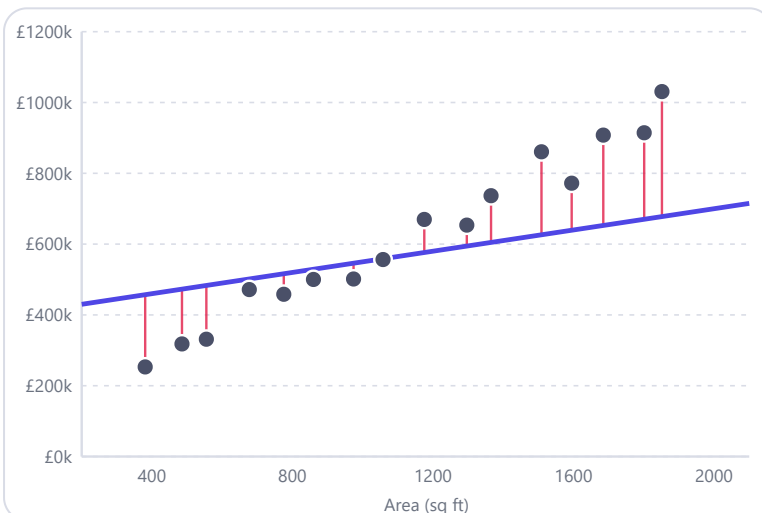


House Price

Picture the data as dots on a chart. **The model tries to find a line that best represents the relationship between the input and target.** "Best" simply means: the line that stays as close as possible to all the dots at once — the one that makes the smallest overall prediction errors.

Be the algorithm: find the best line yourself

SAMPLE DATA



YOUR AVG. ERROR

£136k

BEST POSSIBLE

?☒ Show errors (residuals)

Move the sliders and try to shrink the red error lines.



That's exactly what `LinearRegression().fit()` does — it tries every slope/starting point (cleverly, in one shot) and keeps the line with the smallest total error. You just did it by hand!

**LEADING TO THE NEXT CONCEPT**

"We understand the idea of a best-fit line. Now let's ask Python to find it for us."

HANDS-ON

10

Python: Simple Linear Regression

**WHY ARE WE LEARNING THIS NOW?**

We understand the idea. scikit-learn turns it into 5 lines of code — and the same pattern (`fit` → `predict`) works for almost every model you'll use.

Simple Linear Regression with scikit-learn

PYTHON

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[["area"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

`X = df[["area"]]` Double brackets keep X as a **table** (DataFrame) — scikit-learn always expects a 2-D input, even for one feature.

`y = df["price"]` Single brackets → one column, the target.

`train_test_split( ... )` Hides 20% of rows as a **test set** the model never sees during training — our "exam paper".

`random_state=42` Makes the split repeatable, so everyone in the room gets the same numbers.

`model.fit(X_train, y_train)` Learning happens here — the model finds the best-fit line.

`model.predict(X_test)` Uses the learned line to predict prices for unseen houses.

▼ Peek inside the model (optional)

What did the model learn?

PYTHON

```
print(model.coef_)      # how much price changes per extra sq.ft  
print(model.intercept_) # where the line starts
```

Price is in £ thousands, so a coefficient of 0.45 would mean "each extra sq ft adds about £0.45k (£450) to the price, on average."

**LEADING TO THE NEXT CONCEPT**

"Is house price determined only by area? Of course not. Let's give the model more information."